

Comprehensive Study Notes: Large Language Models (LLMs)

Introduction to LLMs

Large Language Models (LLMs) are deep learning systems designed to understand and generate human language. They are trained on enormous collections of text gathered from books, websites, research papers, documentation, and many other sources. During training, the model learns patterns in language, grammar, semantics, reasoning structures, facts, and contextual relationships. The fundamental objective of an LLM is next-token prediction. Given a sequence of words, the model predicts the most likely token that should come next. Surprisingly, this simple objective enables models to develop capabilities such as summarization, translation, question answering, coding assistance, reasoning, and content generation. Examples of popular LLMs include GPT, Gemini, Claude, and LLaMA.

Tokens and Tokenization

Computers cannot directly understand raw text. Therefore text must be converted into smaller units called tokens. A token may represent: • A complete word • Part of a word (subword) • A punctuation mark • A special symbol Example: 'I am learning about AI' Word tokens: [I] [am] [learning] [about] [AI] Subword tokens: [I] [am] [learn] [ing] [about] [AI] Character tokens: [I] [] [a] [m] [] [] [e] ... Modern LLMs primarily use subword tokenization because it efficiently handles vocabulary size and rare words. Byte Pair Encoding (BPE) and SentencePiece are common tokenization algorithms.

Embeddings

After tokenization, token IDs are transformed into embeddings. An embedding is a dense numerical vector that captures semantic meaning. Instead of treating words as isolated symbols, embeddings represent them in a mathematical space. Example: Cat → [0.21, 0.93, -0.41, ...] Dog → [0.18, 0.88, -0.39, ...] Since cat and dog are semantically related, their embeddings are located close together in vector space. Benefits of embeddings: • Capture semantic similarity • Enable contextual understanding • Reduce sparsity • Improve generalization Embeddings form the foundation of language understanding in transformer models.

How LLMs Work

Step 1: User enters a prompt. Step 2: Tokenization converts text into token IDs. Step 3: Token IDs become embeddings. Step 4: Positional encoding adds information about token order. Step 5: Embeddings pass through transformer layers. Step 6: Self-attention analyzes relationships between tokens. Step 7: Feed-forward networks transform representations. Step 8: Output probabilities are generated. Step 9: The most probable next token is selected. Step 10: Process repeats until a complete response is produced. This iterative prediction process generates coherent text.

Transformer Architecture

Transformers were introduced in the landmark 2017 paper 'Attention Is All You Need'. Major components: 1. Input Embeddings 2. Positional Encoding 3. Multi-Head Self-Attention 4. Feed Forward Neural Networks 5. Residual Connections 6. Layer Normalization 7. Output Layer Advantages: • Parallel processing • Long-range dependency modeling • Scalability • High performance on NLP tasks Modern LLMs are built using stacks of transformer layers.

Attention Mechanism

Attention determines which words are important when interpreting a token. Core concepts: Query (Q): Represents what information a token is seeking. Key (K): Represents available information. Value (V): Represents the actual content. Process: 1. Compute similarity between Query and Keys. 2. Generate attention scores. 3. Apply softmax. 4. Produce weighted combination of Values. Example: Sentence: 'The cat sat on the mat.' When processing 'cat', the model may focus strongly on 'sat' and 'mat' while giving little attention to 'the' or 'on'. This dynamic weighting creates context-aware representations.

Tokens vs Embeddings

Tokens: • Discrete units • Human-readable • Derived from tokenization • Initial representation
Embeddings: • Dense vectors • Numerical representations • Capture semantic meaning • Used for model computation Think of tokens as words and embeddings as mathematical meanings of those words.

Prompt Engineering

Prompt engineering is the practice of designing prompts that guide model behavior. A high-quality prompt includes: 1. Clear Instructions 2. Relevant Context 3. Constraints 4. Desired Format 5. Examples (Few-shot prompting) Weak Prompt: 'Explain AI.' Strong Prompt: 'Explain Artificial Intelligence to a first-year engineering student in 150 words using simple language and three bullet points.' The second prompt provides significantly better guidance.

Prompt Testing and Refinement

Prompt creation is iterative. Workflow: 1. Draft Prompt 2. Test Output 3. Evaluate Results 4. Identify Weaknesses 5. Refine Prompt 6. Retest 7. Compare Versions Evaluation Criteria: • Accuracy • Clarity • Relevance • Completeness • Consistency A/B testing multiple prompts helps identify the best-performing version.

Best Practices

1. Define precise objectives. 2. Avoid ambiguity. 3. Assign roles when appropriate. 4. Specify output format. 5. Provide examples. 6. Iterate and compare results. 7. Measure performance metrics. 8. Use constraints carefully. 9. Validate outputs. 10. Continuously improve prompts. These practices significantly improve reliability and response quality.

Interview and Exam Questions

1. What is an LLM? 2. What is tokenization? 3. Why are embeddings important? 4. Explain transformer architecture. 5. What is self-attention? 6. Differentiate tokens and embeddings. 7. What are Query, Key, and Value vectors? 8. Explain prompt engineering. 9. What are transformer advantages? 10. Describe the LLM text generation process. These are commonly asked in AI, NLP, and Generative AI interviews.

Quick Revision Sheet

LLM = Large Language Model Pipeline: Text → Tokens → Embeddings → Transformer Layers → Attention → Output Probabilities → Generated Text Key Terms: • Tokenization • Embeddings • Positional Encoding • Self-Attention • Transformer • Prompt Engineering • Few-Shot Learning • Zero-Shot Learning Remember: Tokens represent text. Embeddings represent meaning. Attention captures relationships. Transformers power modern LLMs. Prompt engineering improves output quality.